

인코더와 디코더는 각각 입력 어휘 사전 크기(input_vocab_size), 타겟 어휘 사전 크기(target_vocab_size), 레이어 수(num_layers), 모델 차원(d_model), 어텐션 헤드 수(num_heads), 피드포워드 네트워크 차원(d_ff), 드롭아웃 비율(dropout_rate)을 기반으로 설정됩니다.

call 메소드는 모델의 입력 데이터 inputs를 받아 처리합니다. 입력은 컨텍스트(context)와 타겟 시퀀스(x)로 구성됩니다. 먼저 인코더를 통해 컨텍스트를 처리하고, 그 결과를 디코더에 전달하여 타겟 시퀀스를 처리합니다. 최종적으로, Dense 레이어(self.final_layer)를 통해 최종 출력(logits)을 계산합니다.

```
class Transformer(tf.keras.Model): (parameter) d_model: Any
    def __init__(self, *, num_layers, d_model, num_heads, dff,
                  input_vocab_size, target_vocab_size, dropout_rate=0.1):
        super().__init__()
        # 인코더 초기화
        self.encoder = Encoder(num_layers=num_layers, d_model=d_model,
                               num_heads=num_heads, dff=dff,
                               vocab_size=input_vocab_size,
                               dropout_rate=dropout_rate)
```

(dropout_rate)를 기반으로 설정됩니다.

call 메소드는 모델의 입력 데이터 inputs를 받아 처리합니다. 입력은 컨텍스트(context)와 타겟 시퀀스(x)로 구성됩니다. 먼저 인코더를 통해 컨텍스트를 처리하고, 그 결과를 디코더에 전달하여 타겟 시퀀스를 처리합니다. 최종적으로, Dense 레이어(self.final_layer)를 통해 최종 출력(logits)을 계산합니다.

```
class Transformer(tf.keras.Model):
    def __init__(self, *, num_layers, d_model, num_heads, dff,
                  input_vocab_size, target_vocab_size, dropout_rate=0.1):
        super().__init__()
        # 인코더 초기화
        self.encoder = Encoder(num_layers=num_layers, d_model=d_model,
                               num_heads=num_heads, dff=dff,
                               vocab_size=input_vocab_size,
                               dropout_rate=dropout_rate)

        # 디코더 초기화
        self.decoder = Decoder(num_layers=num_layers, d_model=d_model,
                               num_heads=num_heads, dff=dff,
                               vocab_size=target_vocab_size,
```



```
# 최종 출력 반환
return logits
```

Hyperparameters

이 예제를 작고 상대적으로 빠르게 유지하기 위해 레이어 수(`num_layers`), 임베딩의 차원(`d_model`), `FeedForward` 레이어의 내부 차원(`dff`)을 줄입니다. 원본 Transformer 문서에 설명된 기본 모델은 `num_layers=6`, `d_model=512` 및 `dff=2048`을 사용했습니다.

self-attention 헤드의 수는 동일하게 유지됩니다(`num_heads=8`).

```
num_layers = 4
d_model = 128
dff = 512
num_heads = 8
dropout_rate = 0.1
```

마이크 및 카메라 버튼을 클릭하여 다시 시도하거나 브라우저 주소 표시줄에서 마이크 및 카메라 액세스를 활성화하고 페이지를 새로 고침하십시오. [더 알아보기](#)

말하기: aica estsoft

Transformer.ipynb

F: > 관주_강의자료 > 딥러닝_pytorch_강의자료 > 감성소스자료 > NLP > Transformer.ipynb > M4 130. Neural machine translation with a Transformer and Keras > M4 Transformer > class Transformer(tf.keras.Model):

Generate + Code + Markdown Run All Outline

Select Kernel

dropout_rate=dropout_rate)

최종 출력을 위한 Dense 레이어

self.final_layer = tf.keras.layers.Dense(target_vocab_size)

def call(self, inputs):

입력: 컨텍스트와 타겟 시퀀스

context, x = inputs

인코더를 통한 컨텍스트 처리

context = self.encoder(context) # (배치 크기, 컨텍스트 길이, d_model)

디코더를 통한 타겟 시퀀스 처리

x = self.decoder(x, context) # (배치 크기, 타겟 길이, d_model)

최종 출력 계산

logits = self.final_layer(x) # (배치 크기, 타겟 길이, 타겟 어휘 크기)

필요 시 케라스 마스크 제거

try:

del logits.keras_mask

Python

Restricted Mode 0/2

Spaces 2 Sign In Cell 150 of 209 Prettier

self-attention 헤드의 수는 동일하게 유지됩니다(`num_heads=8`).

```
num_layers = 4
d_model = 128
dff = 512
num_heads = 8
dropout_rate = 0.1
```

'Transformer' 모델을 인스턴스화합니다.

```
# 트랜스포머 모델 인스턴스 생성
transformer = Transformer(
    num_layers=num_layers,          # 인코더 및 디코더 레이어 수
    d_model=d_model,                # 임베딩 차원
    num_heads=num_heads,            # 어텐션 헤드의 수
    dff=dff,                        # 피드포워드 네트워크의 차원
    input_vocab_size=tokenizers.pt.get_vocab_size().numpy(), # 포르투갈어 어휘 사전 크기
    target_vocab_size=tokenizers.en.get_vocab_size().numpy(), # 영어 어휘 사전 크기
    dropout_rate=dropout_rate)      # 드롭아웃 비율
```

```
dropout_rate=dropout_rate)
```

```
# 드롭아웃 비율
```

Test

```
# 트랜스포머 모델에 포르투갈어와 영어 입력 적용
```

```
output = transformer((pt, en))
```

```
# 입력 및 출력 형상 출력
```

```
print(en.shape) # 영어 입력 데이터 형상
```

```
print(pt.shape) shape: Any 이터 형상
```

```
print(output.shape) # 트랜스포머 모델 출력 형상
```

```
(64, 128)
```

```
(64, 128)
```

```
(64, 128, 7010)
```

```
# 트랜스포머 모델의 디코더에서 마지막 디코더 레이어의 어텐션 점수 확인
```

```
attn_scores = transformer.decoder.dec_layers[-1].last_attn_scores
```

```
print(attn_scores.shape) # (배치 크기, 어텐션 헤드 수, 타겟 시퀀스 길이, 입력 시퀀스 길이)
```


=====

Total params: 10184162 (38.85 MB)
Trainable params: 10184162 (38.85 MB)
Non-trainable params: 0 (0.00 Byte)

Training

optimizer 설정

원래 Transformer ^I논문의 공식에 따라 사용자 정의 학습률 스케줄러와 함께 Adam 최적화 프로그램을 사용합니다.

$$lrate = d_{model}^{-0.5} * \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1})$$

초기에는 학습률을 서서히 증가시키는 "웜업(warmup)" 기간을 거치며, 이후 학습률을 점차 감소시킵니다.

```
class CustomSchedule(tf.keras.optimizers.schedules.LearningRateSchedule):
```

마이크 및 카메라 버튼을 클릭하여 다시 시도하거나 브라우저 주소 표시줄에서 마이크 및 카메라 액세스를 활성화하고 페이지를 새로 고침하십시오. [더 알아보기](#)

말하기: aica estsoft

Transformer.ipynb

F: > 관주_강의자료 > 딥러닝_pytorch_강의자료 > 강의소스자료 > NLP > Transformer.ipynb > M4 130. Neural machine translation with a Transformer and Keras > M4 Training > M4 model Train > %time

Generate + Code + Markdown Run All Outline

Select Kernel

트랜스포머 모델 컴파일

```
transformer.compile(  
    loss=masked_loss,          # 마스킹된 손실 함수  
    optimizer=optimizer,      # 최적화 알고리즘  
    metrics=[masked_accuracy]  # 마스킹된 정확도를 평가 지표로 사용
```

Python

%time

모델 훈련

```
transformer.fit(train_batches,  
               epochs=20,  
               validation_data=val_batches)
```

Python

Epoch 1/20

290/810 [=====>.....] - ETA: 1:14:53 - loss: 8.0780 - masked_accuracy: 0.0712

추론 실행

마이크 및 카메라 버튼을 클릭하여 다시 시도하거나 브라우저 주소 표시줄에서 마이크 및 카메라 액세스를 활성화하고 페이지를 새로 고침하십시오. [더 알아보기](#)

말하기: aica estsoft

Transformer.ipynb

F: > 광주_강의자료 > 딥러닝_pytorch_강의자료 > 강의소스자료 > NLP > Transformer.ipynb > M4 130. Neural machine translation with a Transformer and Keras > M4 Training > M4 model Train > %time

Generate + Code + Markdown Run All Outline

Select Kernel

트랜스포머 모델 컴파일

```
transformer.compile(  
    loss=masked_loss,  
    optimizer=optimizer,  
    metrics=[masked_accuracy])
```

마스킹된 손실 함수

최적화 알고리즘

마스킹된 정확도를 평가 지표로 사용

Generate + Code + Markdown

Python

%time

모델 훈련

```
transformer.fit(train_batches,  
                epochs=20,  
                validation_data=val_batches)
```

Python

Epoch 1/20

290/810 [=====>.....] - ETA: 1:14:53 - loss: 8.0780 - masked_accuracy: 0.0712

추론 실행

```
translator = Translator(tokenizers, transformer)
```

함수는 세 가지 인자를 받습니다:

sentence: 번역할 원문 문장

tokens: 토큰화된 번역 결과입니다. 이 값은 numpy 배열로 변환되고 UTF-8로 디코딩됩니다.

ground_truth: 실제 번역 결과

```
def print_translation(sentence, tokens, ground_truth):
```

입력 문장 출력

```
print(f'{"Input":15s}: {sentence}')
```

모델의 예측 번역 출력

```
print(f'{"Prediction":15s}: {tokens.numpy().decode("utf-8")}')
```

실제 번역 (ground truth) 출력

```
print(f'{"Ground truth":15s}: {ground_truth}')
```

Example 1:

포르투갈어 문장

```
sentence = 'este é um problema que temos que resolver.'
```

실제 번역 (ground truth)

마이크 및 카메라 버튼을 클릭하여 다시 시도하거나 브라우저 주소 표시줄에서 마이크 및 카메라 액세스를 활성화하고 페이지를 새로 고침하십시오. [더 알아보기](#)

말하기: aica estsoft

```
# 포르투갈어 문장
sentence = 'este é um problema que temos que resolver.'
# 실제 번역 (ground truth)
ground_truth = 'this is a problem we have to solve .'

# Translator를 사용하여 포르투갈어 문장 번역
translated_text, translated_tokens, attention_weights = translator(
    tf.constant(sentence))

# 번역 결과 출력
print_translation(sentence, translated_text, ground_truth)
```

```
Input:      : este é um problema que temos que resolver.
Prediction  : this is a problem that we have to solve .
Ground truth : this is a problem we have to solve .
```

Example 2:

```
sentence = 'os meus vizinhos ouviram sobre esta ideia.'
ground_truth = 'and my neighboring homes heard about this idea .'
```

Spaces: 4

Sign In

Cell 182 of 209

Prettier

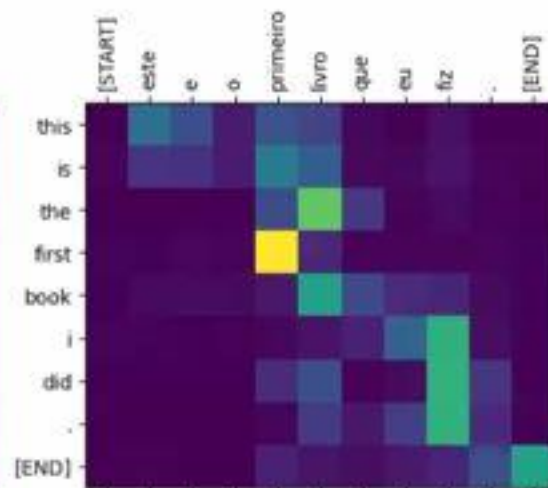
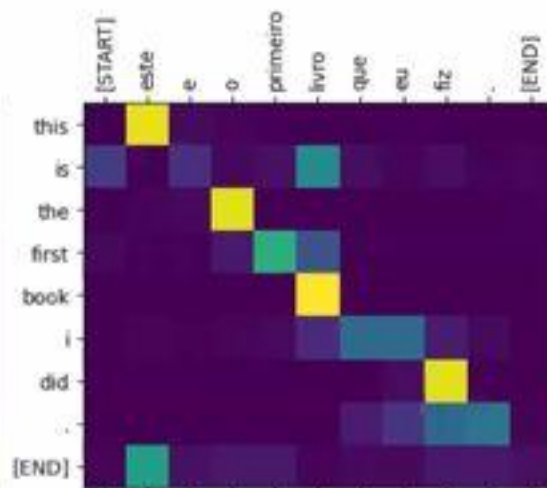
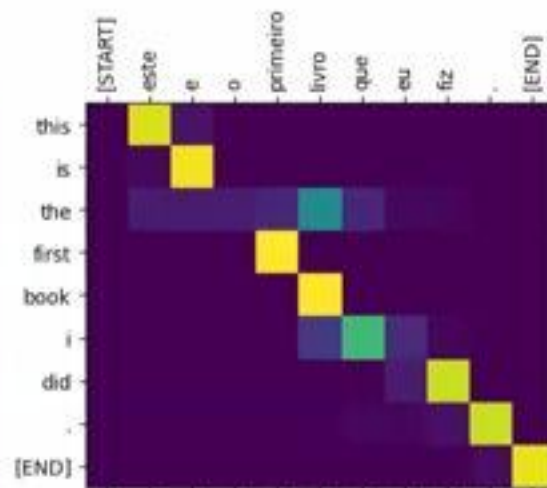
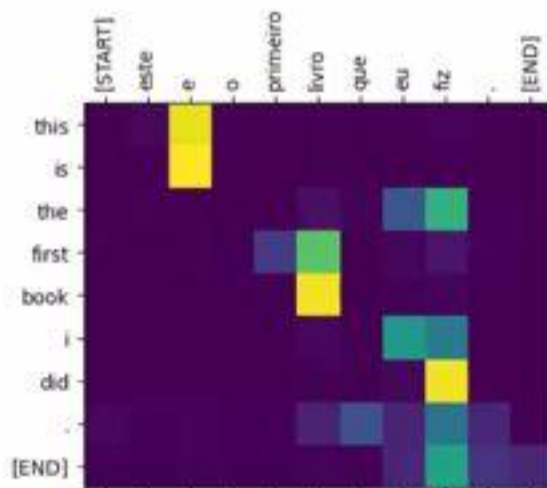
Python

오전 10:11
2025-06-25

```
ax.set_xlabel(f'Head {h+1}') # x축 레이블 설정
```

```
plt.tight_layout() # 그래프 레이아웃 조정  
plt.show() # 그래프 표시
```

```
# 주어진 문장, 번역된 토큰, 첫 번째 어텐션 헤드의 가중치를 사용하여 어텐션 맵 시각화  
plot_attention_weights(sentence,  
                        translated_tokens,  
                        attention_weights[0]) # 첫 번째 어텐션 헤드의 가중치
```



마이크 및 카메라 버튼을 클릭하여 다시 시도하거나 브라우저 주소 표시줄에서 마이크 및 카메라 액세스를 활성화하고 페이지를 새로 고침하십시오. [더 알아보기](#)

말하기: aica estsoft

Restricted Mode is intended for safe code browsing. Trust this window to enable all features.

Transformer.ipynb

F > 강좌_강의자료 > 딥러닝_pytorch_강의자료 > 강의소스자료 > NLP > Transformer.ipynb > M4 130. Neural machine translation with a Transformer and Keras > M4 주론 실행 > sentence = 'os meus vizinhos ouviram sobre esta ideia.'

Generate + Code + Markdown Run All Outline

Select Kernel

```
b'this is the first book i did .'
```

```
tf.saved_model.save(translator, export_dir='translator')
```

Python

```
WARNING:tensorflow:Model's `__init__` arguments contain non-serializable objects. Please implement a `get_config`
```

```
WARNING:tensorflow:Model's `__init__` arguments contain non-serializable objects. Please implement a `get_config`
```

```
reloaded = tf.saved_model.load('translator')
```

Python

```
reloaded('este é o primeiro livro que eu fiz.').numpy()
```

Python

```
b'this is the first book i did .'
```

I

Python

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

huggingface.co/google-bert/bert-base-uncased

Models · Datasets · Spaces · Buckets · Docs · Enterprise · Pricing

Hugging Face is way more fun with friends and colleagues! Join an organization

google-bert / bert-base-uncased

Fill-Mask · Transformers · PyTorch · TensorFlow · JAX · Rust · Core ML · ONNX · Safetensors · bookcorpus · wikipedia · English · bert · exbert · arxiv:1810.04805

License: apache-2.0

Model card · Files and versions · Community

BERT base model (uncased)

Pretrained model on English language using a masked language modeling (MLM) objective. It was introduced in this paper and first released in this repository. This model is uncased: it does not make a difference between english and English.

Disclaimer: The team releasing BERT did not write a model card for this model so this model card has been written by the Hugging Face team.

Model description

BERT is a transformers model pretrained on a large corpus of English data in a self-supervised fashion. This means it was pretrained on the raw texts only, with no humans labeling them in any way (which is why it can use lots of publicly available data) with an automatic process to generate inputs and labels from those texts. More precisely, it was pretrained with two objectives:

Masked language modeling (MLM): taking a sentence, the model randomly masks 15% of the

Downloads last month

60,462,664

Safetensors

Model size 0.1B params

Tensor type F32

Inference Providers

HF Inference API

Fill-Mask

Mask token: [MASK]

Your prompt here...

Generate

View Code Snippets

오전 10:29

2025-06-25

huggingface.co/google-bert/bert-base-uncased

Google DriveNAVERDaumGoogleYouTubePapagoGeminiChatGPTPerplexityClaude아구YES24KOSA4만: AI융합4만: AI융합_데아...

token': 4827,
token_str': 'fashion'},
{ 'sequence': "[CLS] hello i'm a role model. [SEP]",
score': 0.00774490654468536,
token': 2535,
token_str': 'role'},
{ 'sequence': "[CLS] hello i'm a new model. [SEP]",
score': 0.05338378623127937,
token': 2047,
token_str': 'new'},
{ 'sequence': "[CLS] hello i'm a super model. [SEP]",
score': 0.04667217284448994,
token': 3565,
token_str': 'super'},
{ 'sequence': "[CLS] hello i'm a fine model. [SEP]",
score': 0.027895865458250046,
token': 2986,
token_str': 'fine'}}

Here is how to use this model to get the features of a given text in PyTorch:

```
from transformers import BertTokenizer, BertModel  
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')  
model = BertModel.from_pretrained("bert-base-uncased")  
text = "Replace me by any text you'd like."  
encoded_input = tokenizer(text, return_tensors='pt')  
output = model(**encoded_input)
```

and in TensorFlow:

```
from transformers import BertTokenizer, TFBertModel
```

이 페이지에 대해 질문하기

알리기: aica estsoft

아이크 및 카메라 버튼을 클릭하여 다시 시도하거나 브라우저 주소 표시줄에서 마이크 및 카메라 액세스를 활성화하고 페이지를 새로 고침하십시오. [더 알아보기](#)

오전 10:30
2025-06-25

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

I

Transfer Learning

- ✓ 기존에 학습된 모델의 지식을 새로운 문제에 재사용하는 머신러닝 기법
- ✓ 충분한 데이터가 없는 경우, 사전 학습된 모델을 활용해 성능 향상 시킴

I

Transfer Learning

✓ Training Time 절감

- GPT-3 와 같은 최첨단 model 은 train 에 약 2개월 소요

✓ 예측 성능 향상

✓ Small dataset 으로 첨단 performance 구현

Transfer Learning

✓ Training Time 절감

- GPT-3 와 같은 최첨단 model 은 train 에 약 2개월 소요

✓ 예측 성능 향상

✓ Small dataset 으로 첨단 performance 구현

Transfer Learning

✓ Training Time 절감

- GPT-3 와 같은 최첨단 model 은 train 에 약 2개월 소요

✓ 예측 성능 향상

✓ Small dataset 으로 첨단 performance 구현

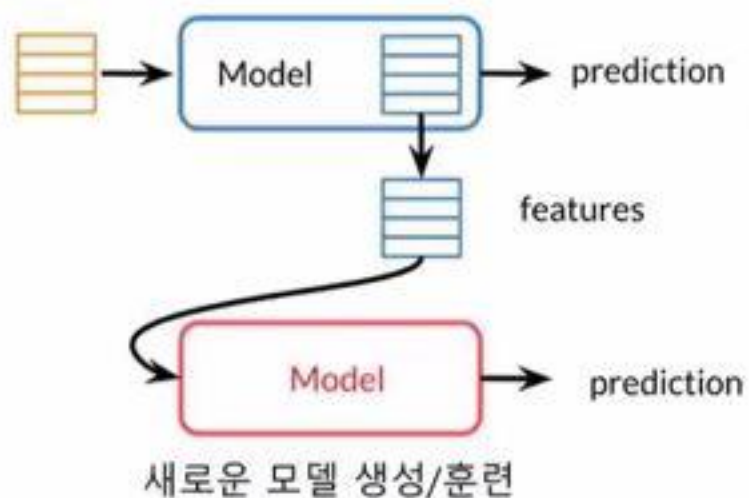
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

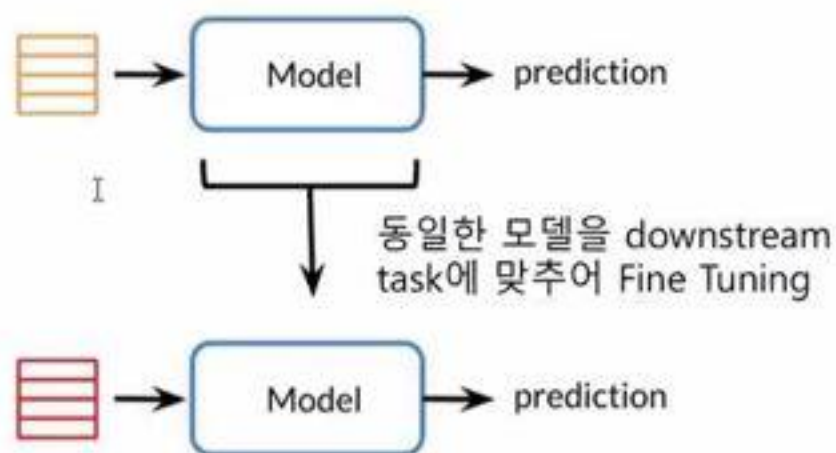
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



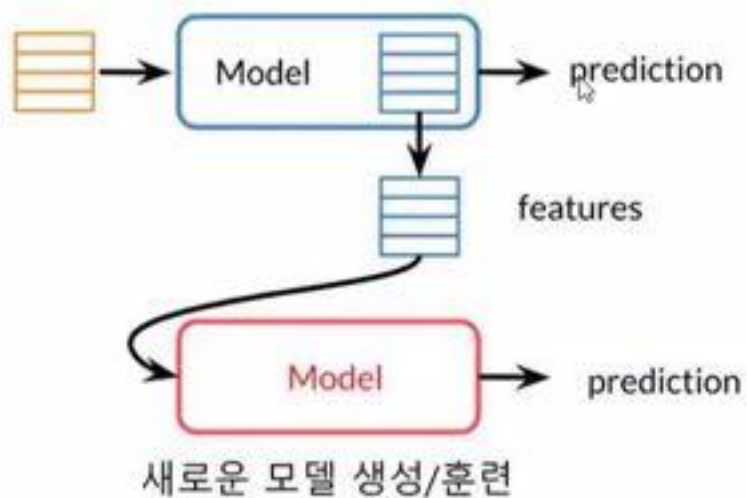
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

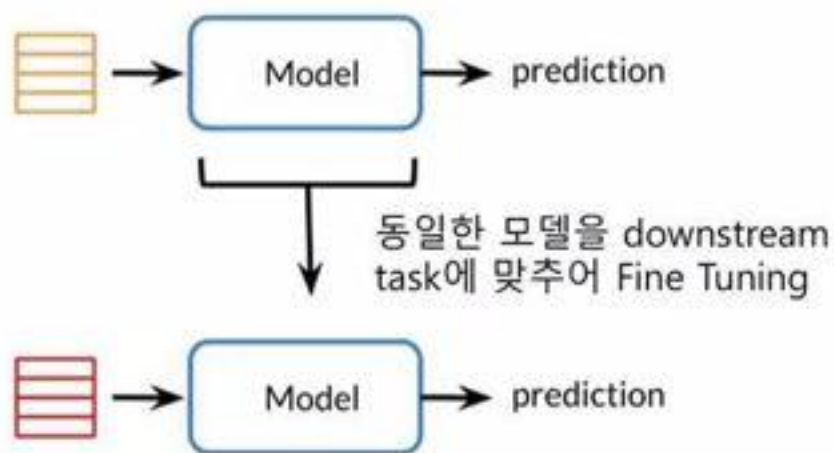
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



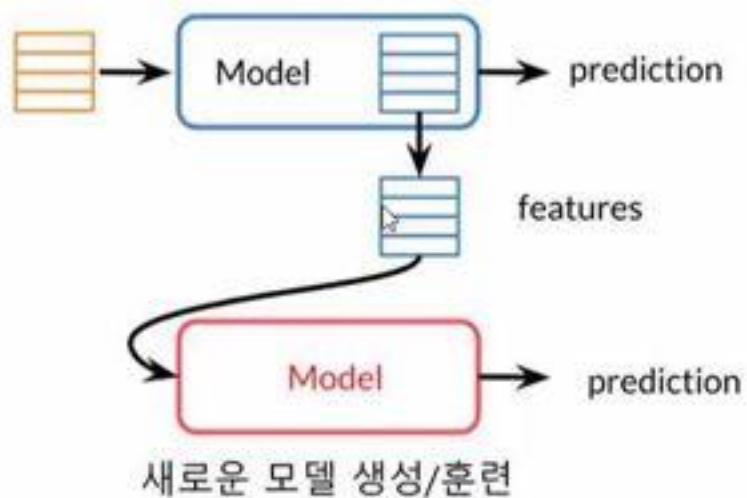
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

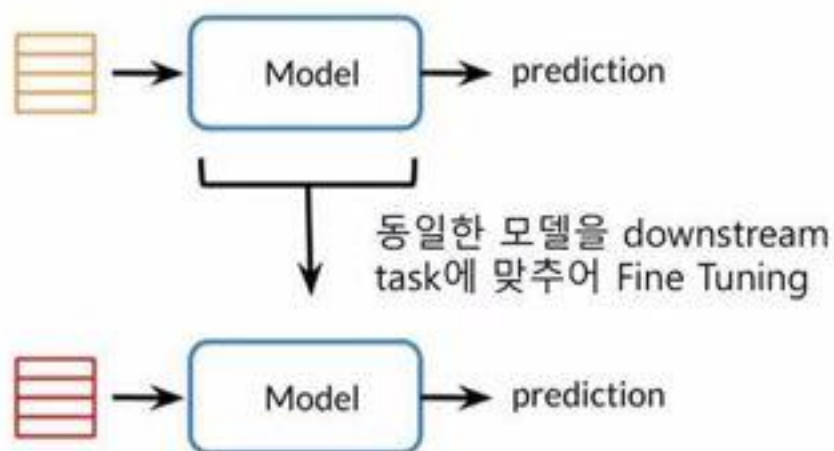
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



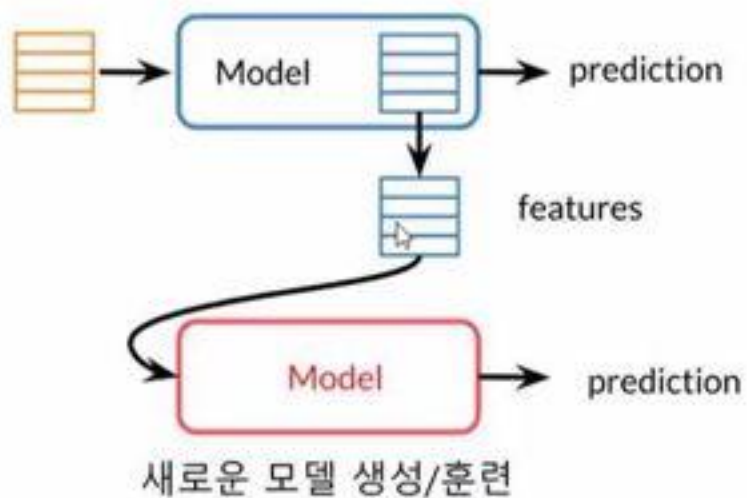
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

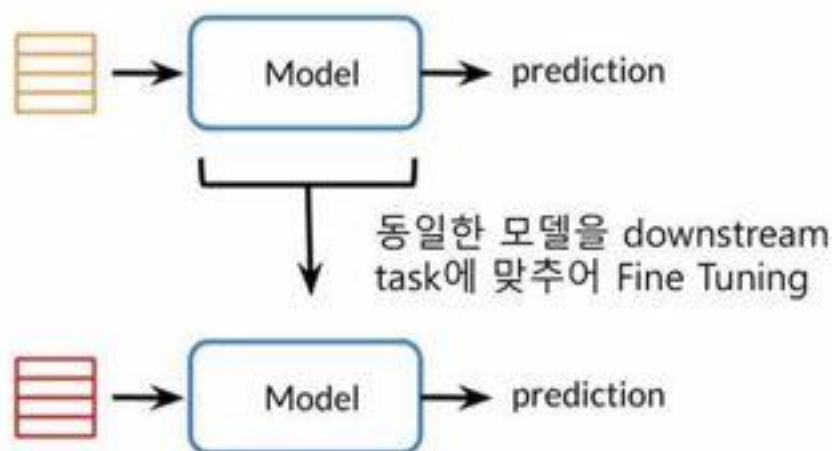
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



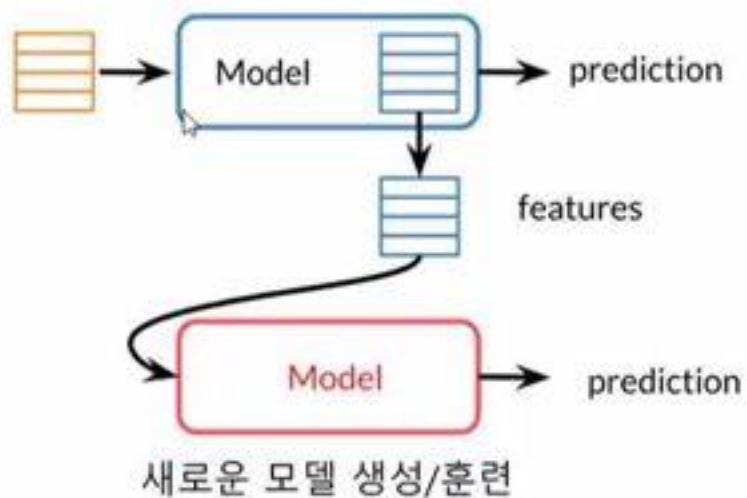
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

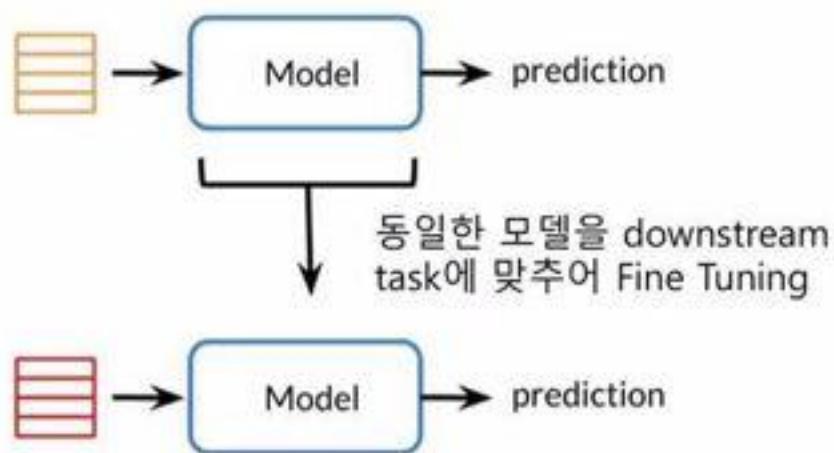
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



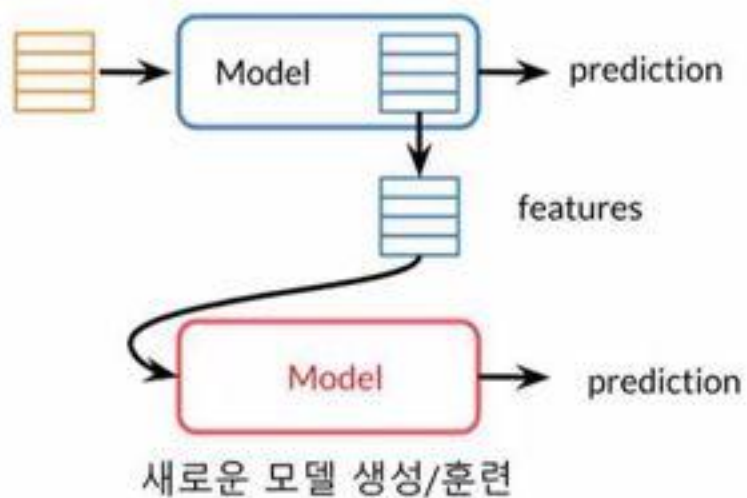
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

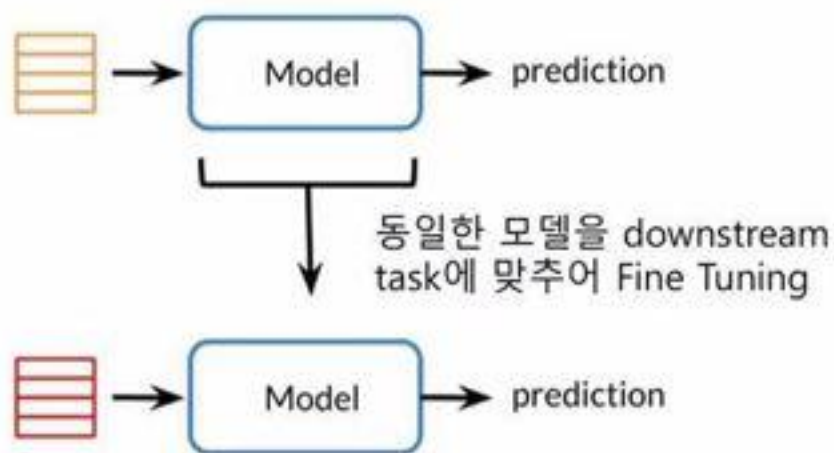
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



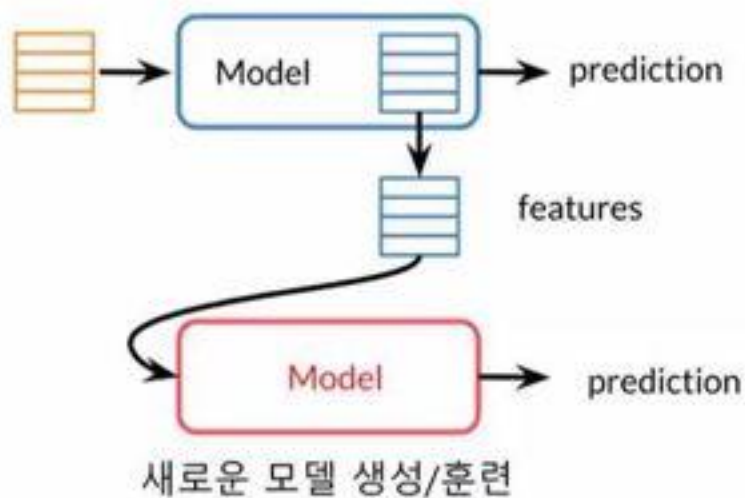
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

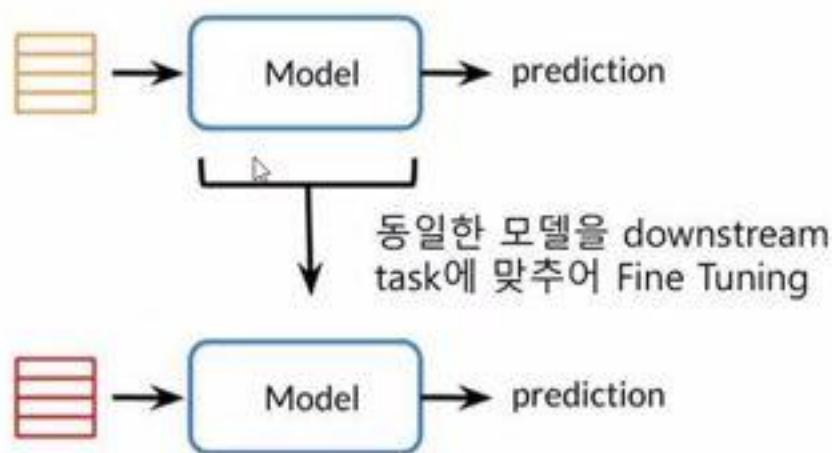
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



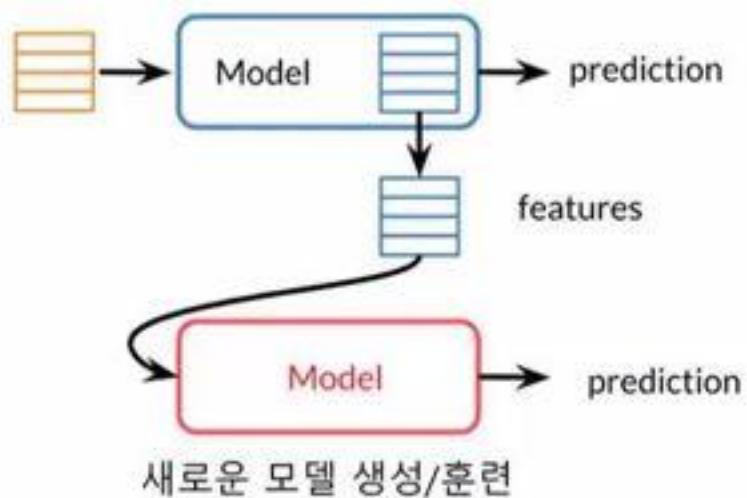
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

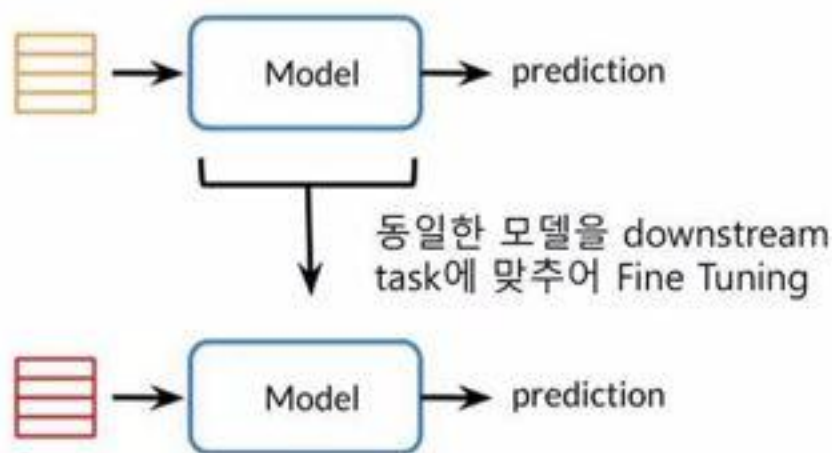
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



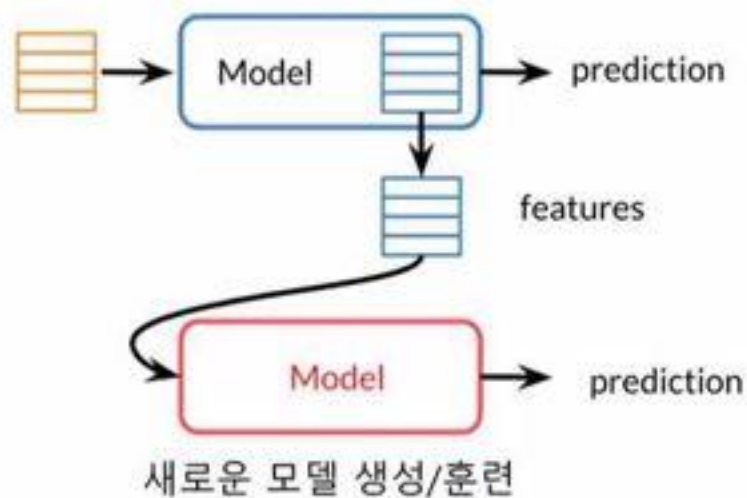
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

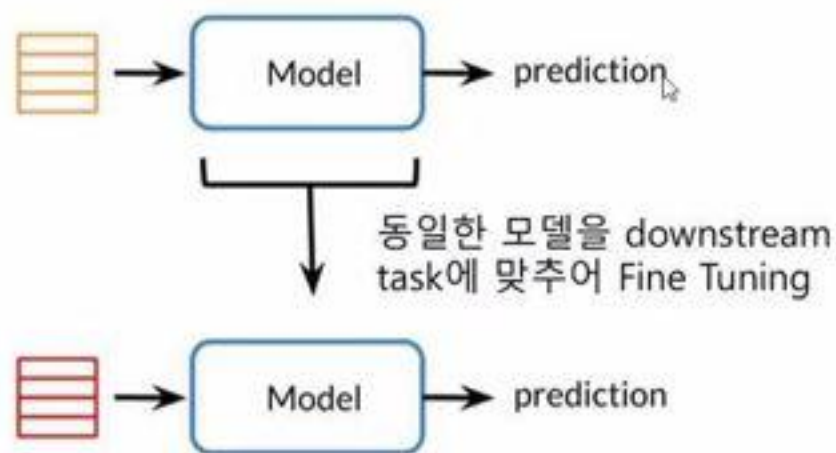
Pre-Train



Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



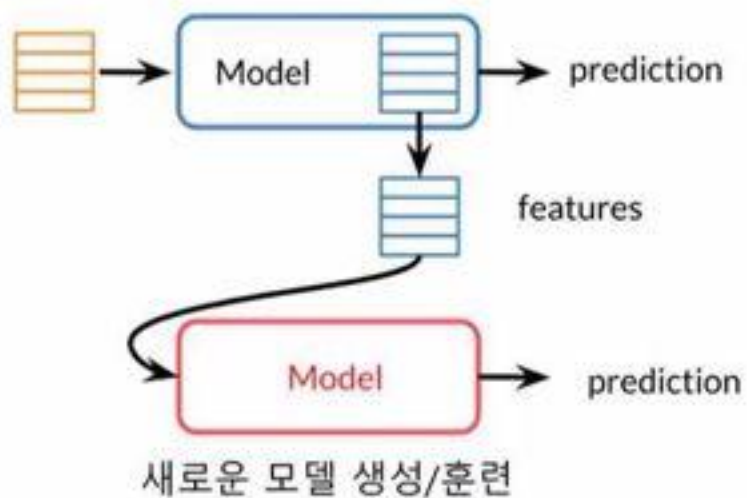
Transfer Learning

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

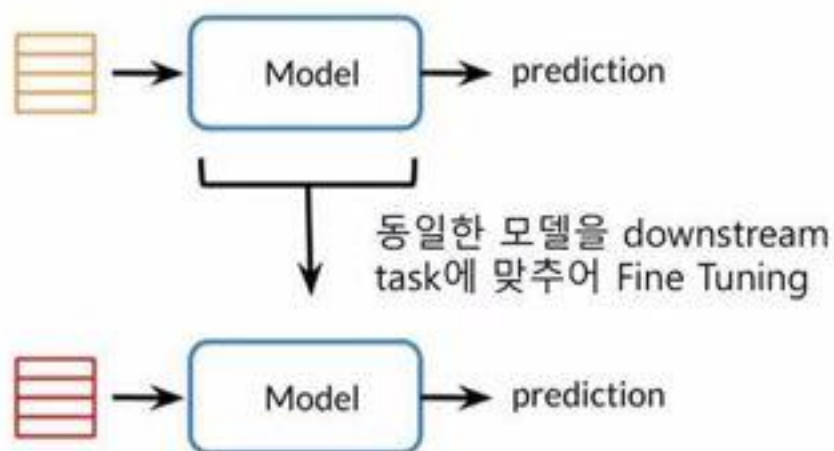
Pre-Train



Fine-Tuning

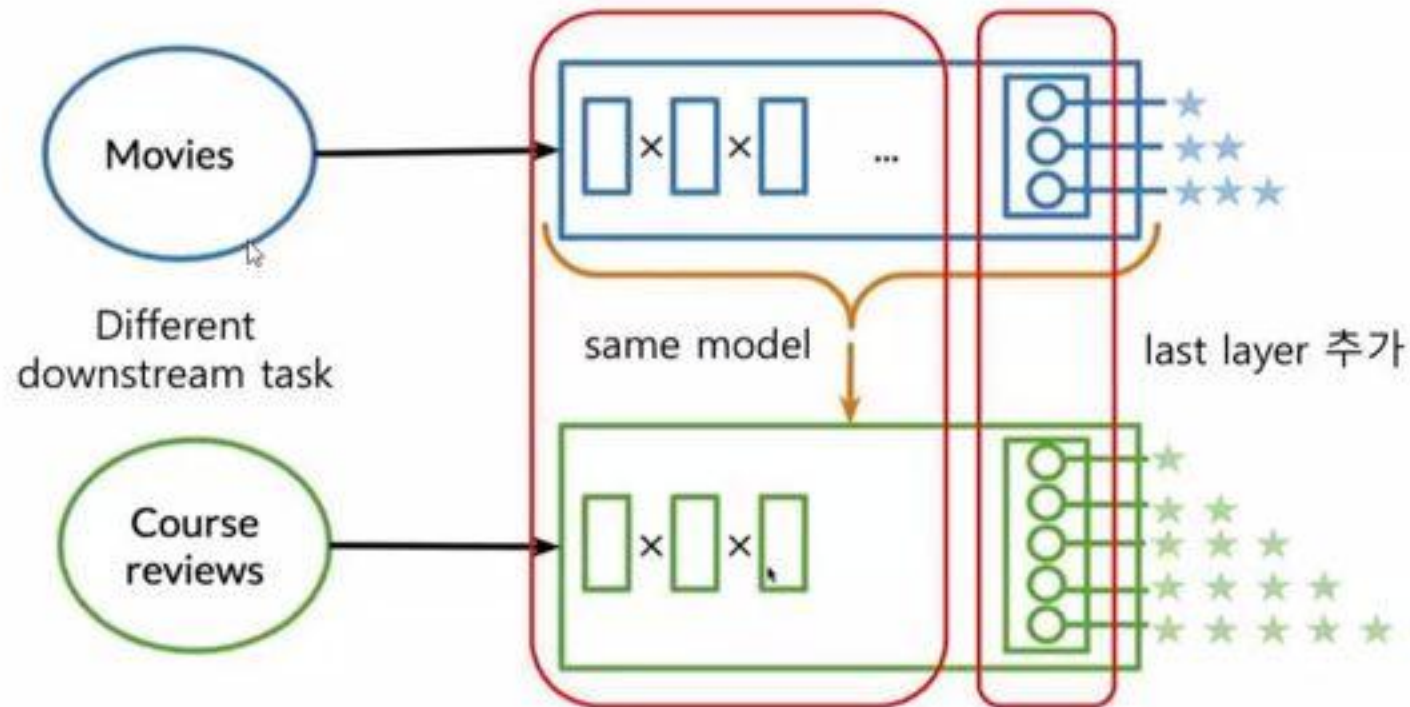
pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



Transfer Learning

Fine-tuning – last layer 추가



- Large size
- Slow fine-tuning
- Slow inferencing
- 특정 domain 에 특화된 단어를 모름



Embedding Layer	~24M
Transformers (x12)	~7M ea. x 12 = ~85M
TOTAL	~109M
	(417 MB)

- 58 -

BERT

BERT

- 2018. Nov. 발표 (<https://github.com/google-research/bert>)
 - 104 개 multi-language model 동시 발표
 - ALBERT, RoBERTA, TinyBERT, DistilBERT, and SpanBERT 등 다양한 변종

BERT

- 2018. Nov. 발표 (<https://github.com/google-research/bert>)
 - 104 개 multi-language model 동시 발표
 - ALBERT, RoBERTA, TinyBERT, DistilBERT, and SpanBERT 등 다양한 변종
- 질의 응답 시스템, 감성분석등의 다양한 GLUE task 에서 SOTA 달성
- Pre-trained token embedding (WordPiece) 사용
 - 30,000(영문 version) / 110,000(104개국어 version) subwords
- BERT-Base model : Transformer layer 12 개, Total parameters 110M
- BERT-Large model : Transformer layer 24 개, Total parameters 340M

BERT

BERT 의 접근 방식

- 1) 범용 Solution 을 (Transformer 의 encoder 이용)
- 2) Scalable 한 형태로 구현하여
- 3) 많은 머신 리소스로 훈련하여 성능을 높인다.
 - Pre-training 에 사용한 data
 - BooksCorpus (800M 단어)
 - English Wikipedia (2500M 단어)
 - BERT-Base : 64 TPU 4 days 소요
 - 11 개의 NLP 과제에 하나의 pre-trained model 을 공통적으로 사용. 단, 각각의 과제에 대해 약간의 fine-tuning 적용
 - 발표 당시 SQuAD(Stanford Question Answering Dataset) 에서 human level 능가
 - NLP 를 위한 AI solution 개발은 coding 능력을 갖춘 모든 사람에게 개방

BERT 의 접근 방식

- 1) 범용 Solution 을 (Transformer 의 encoder 이용)
- 2) Scalable 한 형태로 구현하여
- 3) 많은 머신 리소스로 훈련하여 성능을 높인다.
 - Pre-training 에 사용한 data
 - BooksCorpus (800M 단어)
 - English Wikipedia (2500M 단어)
 - BERT-Base : 64 TPU 4 days 소요
 - 11 개의 NLP 과제에 하나의 pre-trained model 을 공통적으로 사용. 단, 각각의 과제에 대해 약간의 fine-tuning 적용
 - 발표 당시 SQuAD(Stanford Question Answering Dataset)에서 human level 능가
 - NLP 를 위한 AI solution 개발은 coding 능력을 갖춘 모든 사람에게 개방

nuggingface

Hugging Face (https://github.com/huggingface)

- NLP 와 관련된 다양한 패키지를 제공

Search or jump to... Pull requests Issues Marketplace Explore

 **Hugging Face**
The AI community building the future.
NYC + Paris <https://huggingface.co/> Verified

Overview Repositories 52 Packages People 26 Projects Sponsoring 4

Pinned

 transformers Transformers: State-of-the-art Natural Language Processing for PyTorch, TensorFlow, and JAX. Python ☆ 50.4k ▼ 12k	 datasets The largest hub of ready-to-use datasets for ML models with fast, easy-to-use and efficient data manipulation tools. Python ☆ 8.8k ▼ 1.1k	 tokenizers Fast State-of-the-Art Tokenizers optimized for Research and Production. Rust ☆ 4.8k ▼ 375
 knockknock Knock Knock: Get notified when your training ends with only two additional lines of code. Python ☆ 2.2k ▼ 130	 accelerate A simple way to train and use PyTorch models with multi-GPU, TPU, mixed-precision. Python ☆ 1.8k ▼ 89	 huggingface_hub all the open source things related to the Hugging Face Hub. Python ☆ 196 ▼ 34

HuggingFace

Hugging Face

- Transformers
 - Transformer 기반 (masked) language model 알고리즘 제공
 - 기 학습된 (pretrained) 모델 배포
- Tokenizers
 - transformers package 에서 사용할 수 있는 subword 토큰나이저 학습
 - transformers 와 분리되어 다른 목적에도 이용할 수 있다.
- dataset → tokenizer → models 의 언어 모델 학습에 필요한 전 과정을 지원

HuggingFace

Hugging Face

- Transformers
 - Transformer 기반 (masked) language model 알고리즘 제공
 - 기 학습된 (pretrained) 모델 배포
- Tokenizers
 - transformers package 에서 사용할 수 있는 subword 토큰나이저 학습
 - transformers 와 분리되어 다른 목적에도 이용할 수 있다.
- dataset → tokenizer → models 의 언어 모델 학습에 필요한 전 과정을 지원